DES – Data Encryption Standard

Tables Reference

Step 1 : Initial Permutation (IP)

Purpose: Rearranges all 64 plaintext bits to new positions before the Feistel rounds. This provides no cryptographic strength asit was added for hardware implementation reasons.

How it works

Output bit i is taken from input bit IP[i]. The result is split into two 32-bit halves:

```
IPout[i] = Plaintext[ IP[i] ]    for i = 1 to 64
L0 = IPout[1..32]    R0 = IPout[33..64]
```

IP Table (64-bit input → 64-bit output, 8 columns)

Each cell shows which input bit maps to that output position (row-major order, output bit 1 first):

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Step 2a : PC-1: Permuted Choice 1 (Key Reduction 64 → 56 bits)

Purpose: Removes the 8 parity bits from the 64-bit master key (positions 8, 16, 24, 32, 40, 48, 56, 64), yielding 56 usable key bits.

How it works

```
PC1out[i] = Key[ PC1[i] ]    for i = 1 to 56
C0 = PC1out[1..28]    D0 = PC1out[29..56]
```

PC-1 Table (7 columns × 8 rows = 56 entries)

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

Step 2b : Left Shift (Rotation) Schedule

Purpose: Each round, C and D are independently left-rotated by the number of positions shown below. This ensures every subkey uses different key bits.

How it works

```
Ci = LeftRotate( C(i-1), shifts[i] )
Di = LeftRotate( D(i-1), shifts[i] )
```

Shift Schedule Table (16 rounds)

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Shifts	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

Step 2c : PC-2: Permuted Choice 2 (Subkey Compression 56 → 48 bits)

Purpose: Selects and permutes 48 of the 56 bits from the combined Ci||Di halves to form the 48-bit round subkey Ki. Eight bits of CD are discarded each round.

How it works

```
Ki = PC2( Ci || Di )     for i = 1 to 16
```

PC-2 Table (6 columns × 8 rows = 48 entries)

14	17	11	24	1	5
3	28	15	6	21	10
23	19	12	4	26	8
16	7	27	20	13	2
41	52	31	37	47	55
30	40	51	45	33	48
44	49	39	56	34	53
46	42	50	36	29	32

Step 3a : E-Bit Expansion (32 → 48 bits)

Purpose: Expands the 32-bit right half R to 48 bits so it can be XOR'd with the 48-bit subkey Ki. Some bits appear twice (at block boundaries) to create 8 groups of 6 bits.

How it works

```
E_out[i] = R[ E[i] ]     for i = 1 to 48
```

Each row of 6 below is one 6-bit group fed into one S-box:

E Expansion Table (6 columns × 8 rows = 48 entries)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

Step 3b : XOR with Subkey

Purpose: Mixes the secret key into the data. This is where the key material directly influences the output.

Formula

```
X = E(R(i-1))  XOR  Ki        (48 bits XOR 48 bits = 48 bits)
```

No table is used here as it is a bitwise XOR operation applied to all 48 bits simultaneously.

Step 3c : S-Box Substitution (48 → 32 bits)

Purpose: The only non-linear step in DES being the core of its security. Prevents linear attacks by making the output non-linearly dependent on the input.

How it works

The 48-bit XOR result X is split into 8 groups of 6 bits. For each group:

```
row = concat( bit_1, bit_6 )        (2-bit index, range 0–3)
col = bits_2_to_5                   (4-bit index, range 0–15)
output = S_box[ row ][ col ]         (4-bit value)
```

Total output: 8 × 4 = 32 bits. Each S-box table below has 4 rows (0–3) and 16 columns (0–15).

S-Box 1 (S1)

r\c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

S-Box 2 (S2)

r\c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	15	1	8	14	6	11	3	4	9	7	2	13	12	0	5	10
1	3	13	4	7	15	2	8	14	12	0	1	10	6	9	11	5
2	0	14	7	11	10	4	13	1	5	8	12	6	9	3	2	15
3	13	8	10	1	3	15	4	2	11	6	7	12	0	5	14	9

S-Box 3 (S3)

r\c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	10	0	9	14	6	3	15	5	1	13	12	7	11	4	2	8
1	13	7	0	9	3	4	6	10	2	8	5	14	12	11	15	1
2	13	6	4	9	8	15	3	0	11	1	2	12	5	10	14	7
3	1	10	13	0	6	9	8	7	4	15	14	3	11	5	2	12

S-Box 4 (S4)

r\c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	7	13	14	3	0	6	9	10	1	2	8	5	11	12	4	15
1	13	8	11	5	6	15	0	3	4	7	2	12	1	10	14	9
2	10	6	9	0	12	11	7	13	15	1	3	14	5	2	8	4
3	3	15	0	6	10	1	13	8	9	4	5	11	12	7	2	14

S-Box 5 (S5)

r\c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	2	12	4	1	7	10	11	6	8	5	3	15	13	0	14	9
1	14	11	2	12	4	7	13	1	5	0	15	10	3	9	8	6
2	4	2	1	11	10	13	7	8	15	9	12	5	6	3	0	14
3	11	8	12	7	1	14	2	13	6	15	0	9	10	4	5	3

S-Box 6 (S6)

r\c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	12	1	10	15	9	2	6	8	0	13	3	4	14	7	5	11
1	10	15	4	2	7	12	9	5	6	1	13	14	0	11	3	8
2	9	14	15	5	2	8	12	3	7	0	4	10	1	13	11	6
3	4	3	2	12	9	5	15	10	11	14	1	7	6	0	8	13

S-Box 7 (S7)

r\c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	4	11	2	14	15	0	8	13	3	12	9	7	5	10	6	1
1	13	0	11	7	4	9	1	10	14	3	5	12	2	15	8	6
2	1	4	11	13	12	3	7	14	10	15	6	8	0	5	9	2
3	6	11	13	8	1	4	10	7	9	5	0	15	14	2	3	12

S-Box 8 (S8)

r\c	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
1	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
2	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
3	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Step 3d : P-Permutation (32 → 32 bits)

Purpose: Spreads the output bits of the 8 S-boxes across all columns, ensuring that each S-box output affects multiple S-boxes in the next round (diffusion / avalanche effect).

How it works

```
P_out[i] = S_out[ P[i] ]    for i = 1 to 32
f(R, K) = P( S1..S8( E(R) XOR K ) )
```

P-Permutation Table (4 columns × 8 rows = 32 entries)

16	7	20	21
29	12	28	17
1	15	23	26
5	18	31	10
2	8	24	14
32	27	3	9
19	13	30	6
22	11	4	25

Step 4 : Feistel Round (Repeated 16 Times)

Each round applies the Feistel structure using the round function f computed from Steps 3a–3d:

```
L(i) = R(i-1)
R(i) = L(i-1)  XOR  f( R(i-1), Ki )
```

After Round 16, the two halves are SWAPPED before the final permutation:

```
Pre-IP-inv = R16 || L16    (R before L - note the swap)
```

Step 5 : Final Permutation / Inverse IP (IP^{-1})

Purpose: Undoes the initial permutation to produce the final 64-bit ciphertext. Applied to R16||L16 (halves already swapped).

How it works

```
Ciphertext[i] = Pre_output[ IP_inv[i] ]    for i = 1 to 64
```

IP^{-1} Table (8 columns × 8 rows = 64 entries)

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25